

Advanced Trading & Execution Platform

Aracı kurumların mevcut emir iletim zincirini bozmadan önüne gelen, kurum bağımsız gelişmiş emir (EMS) katmanı — Osmanlı Aktif Trader'ın üzerine çıkan tasarım

İçindekiler

1. Yönetici Özeti
2. Problem: müşterinin şikâyeti
3. Fon Copilot ile fark
4. Benchmark: Osmanlı, MatriksIQ, TradingView, ATP
5. Ürün konumlandırması
6. Mimari
7. Emir seti: Osmanlı vs. hedef
8. Sentetik emir modeli ve durum makinesi
9. Broker Adapter katmanı
10. Trader UX: workspace ve DOM
11. Teknoloji yığını
12. Kurulan iskelet (bist-ems)
13. Riskler ve uyum
14. Yol haritası

1. Yönetici Özeti

Müşteri, iDeal Data'nın al/sat ekranlarını yetersiz buluyor ve gelişmiş emir araçları (zincir emir, OCO, trailing stop vb.) istiyor. Her aracı kurumun kendi emir iletim ekranı var ve müşteri bu zinciri değiştirmek istemiyor. Doğru ürün, mevcut OMS/BIST bağlantısının **önüne** gelen bir Execution Management System katmanıdır: **Client → Advanced EMS → mevcut OMS → BIST**. Osmanlı Yatırım'ın Matriks tabanlı Aktif Trader'ı iyi bir baseline'dır; bu ürün aynı sentetik emir motorunu kurum bağımsız, white-label ve daha geniş emir setiyle sunar.

Konum

EMS

Zincir emir

Sunucu

Backend

NestJS

Event bus

NATS JS

Kritik ürün kararı

Borsa İstanbul'a yeni emir tipi eklenmez; borsanın native emirlerinin üzerinde sunucu tarafında çalışan bir synthetic order engine kurulur. Tarayıcı kapansa da zincir devam eder.

2. Problem: müşterinin şikâyeti

Mevcut durumda trader, kurumunun emir ekranında yalnızca temel alış/satış yapabiliyor. İstenen şey ekranı güzelleştirmek değil; profesyonel emir orkestrasyonu: parent emir dolunca otomatik kâr al / zarar durdur bağlanması, biri gerçekleşince diğerinin iptali (OCO), kademeli çıkış, hareketli stop, zamana yayılmış icra (TWAP), açık pozisyona sonradan koruma ekleme. Bunların hiçbirisi kurumun mevcut OMS'ini değiştirmeyi gerektirmez.

3. Fon Copilot ile fark

	AI Fon Copilot	Advanced Trading Platform (bu doküman)
Varlık	TEFAS fonları, günlük NAV, T+1/T+2 valör	BIST/VİOP hisse ve türev, milisaniye tick
Karar veren	Quant/AI engine ("hangi fon, ne ağırlık")	İnsan trader; sistem emrin nasıl iletileceğini orkestre eder
İşlem sıklığı	Ayda birkaç	Gün içi yüzlerce
Ağırlık merkezi	Python quant + LangGraph	Order state machine + broker adapter + realtime UI
AI rolü	Merkezi (açıklanabilir öneri)	Yok denecek kadar az
Ortak nokta	Yatırımcı hesabı, Next.js + Tailwind / Expo stack'i, NestJS ürün API'si	

4. Benchmark

Osmanlı Yatırım — Aktif Trader (UX + sentetik emir benchmark'ı)

Matriks altyapısı üzerinde çalışır. Kâr Al / Zarar Durdur, hareketli zarar durdur, 30 güne kadar süreli emir, parçalı emir, şartlı emir, OCO, zincir emir, şarta/zincire bağlı emir iptali ve ViOP strateji emirleri sunar. Kokpit'te tek tık emir, otomatik adet, drag&drop fiyat değiştirme ve aynı ekrandan cancel/replace vardır. TradingView'deki kullanıcı yorumlarında açık emre sonradan stop-loss / take-profit bağlayamama ve trailing-stop beklentisi açıkça görülür — doğrudan üstüne çıkılacak alan.

MatriksIQ — özellik benchmark'ı

TWAP, sepet emir, kademeli emir, pair emir, toplu emir, strateji emirleri, otomatik emir ve algoritmik işlemler; birçok aracı kurum hesabıyla çalışır. iDeal yerine detaylı benchmark olarak bu alınmalıdır.

TradingView Broker Integration — frontend mimari benchmark'ı

Trading UI → Broker API → Broker backend modeli; Order Ticket, Account Manager, place/modify/cancel broker backend'ine bridge edilir. "Mevcut emir altyapısını değiştirmeden yeni işlem UI'ı" modelinin kanıtlanmış örneği.

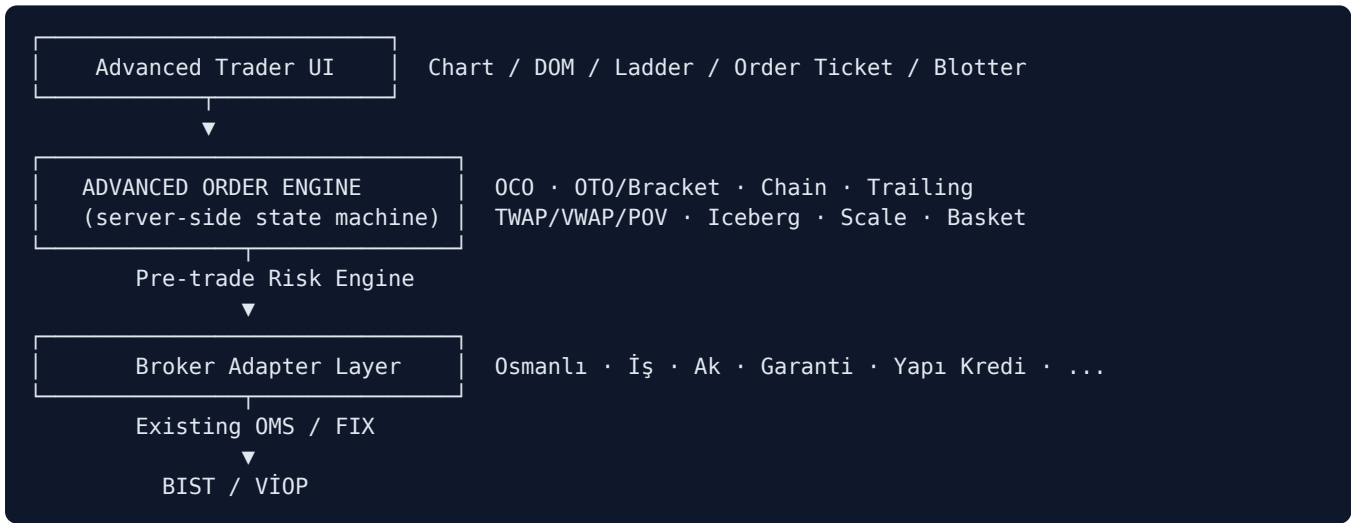
ATP — kurumsal altyapı benchmark'ı

GTP Brokerage / GTP PMS / XFront; aracı kurum bağlantıları, online emir iletimi, pre/post-trade kontrol, yüksek hızlı emir iletimi. Retail UI için değil OMS/backend tarafı için incelenmeli. iDeal çoklu kurum hesabı, algoritmik işlem ve otomatik TP/Stop sunar ama müşterinin şikâyeti UI/EMS deneyimi olduğundan hedef benchmark yapılmamalıdır.

5. Ürün konumlandırması

Aracı kuruma verilen mesaj: **"Mevcut OMS'inizi veya BIST bağlantınızı değiştirmiyoruz. Mevcut emir iletim altyapınızın önüne modern bir execution layer getiriyoruz."** Üç modül: Trading Workspace (chart, DOM, watchlist, positions, orders, P&L), Advanced Order Engine (OCO, OTO, bracket, chain, trailing, TWAP, VWAP, iceberg, basket, scale in/out) ve Broker Gateway (FIX, REST, WebSocket, mevcut kurum API'si). Her kurum için ürün yeniden yazılmaz; yalnızca adapter implementasyonu değişir.

6. Mimari



Zincir kırılmaz: kurum Client → mevcut OMS → BIST kullanmaya devam eder; ürün araya Advanced EMS olarak girer. Runtime akışı:

Browser → Advanced Order API → Order State Machine → PostgreSQL / Redis
→ Event Processor → Broker OMS

7. Emir seti: Osmanlı vs. hedef

Emir	Osmanlı	Hedef	Emir	Osmanlı	Hedef
Limit / Market	var	✓	OTO	yok	✓
Stop Loss / Take Profit	var	✓	OTOCO	yok	✓
Trailing Stop	var	✓	TWAP	platforma bağlı	✓
Conditional	var	✓	VWAP / POV	yok	✓
OCO	var	✓	Iceberg	yok	✓
Chain	var	✓	Scale In / Out	yok	✓
Sliced Order	var	✓	Multi-target TP	yok	✓
Conditional Cancel	var	✓	Break-even Stop	yok	✓
Bracket	kısmen	✓	Mevcut pozisyona SL/TP	sınırlı	✓
Basket	yok	✓	Pair Trading	sınırlı	✓
Order Templates	yok	✓	Multi-account Order	yok	✓
Position Protection	sınırlı	✓	Smart Order Routing	yok	gelecek

8. Sentetik emir modeli ve durum makinesi

Tüm gelişmiş emirler tek bir `AdvancedOrderRequest` ile ifade edilir: leg listesi + `t tree` (parent → child; FILLED veya PARTIAL tetiği) + `ocoGroups`. Bracket = 1 parent + 2 child (TP, SL) aynı OCO grubunda; chain = zincir ağaç; scale-out = 1 parent + N TP + 1 SL; TWAP/VWAP/iceberg = engine içi slicer.

```
Parent   BUY GARAN 1000 @125
├── TP1   SELL 500 @130      } OCO
├── TP2   SELL 500 @135      } grubu
└── STOP  SELL 1000 @121     }
Kural: parent FILLED olmadan child'lar borsaya gitmez.
TP1 500 dolarsa → remaining_position = 500 → stop_qty = 500 otomatik düşer.
```

Durum makinesi (izinli geçişler kodda tablo olarak tutulur, yasadışı geçiş exception fırlatır):

```
DRAFT → PENDING_TRIGGER → PENDING_RISK → SENDING → WORKING → PARTIALLY_FILLED → FILLED
                                     ↳ REJECTED      ↳ CANCELLING → CANCELLED
                                     ↳ ERROR          ↳ EXPIRED
```

STOP / TRAILING / CONDITIONAL leg'ler borsaya gönderilmez; market-data trigger engine tetikleyene kadar `PENDING_TRIGGER` 'da bekler, tetiklenince native LIMIT/MARKET olarak gider.

9. Broker Adapter katmanı

```
interface BrokerAdapter {
  placeOrder() modifyOrder() cancelOrder()
  getOrders() getPositions() getBalances()
  onFill() onOrderUpdate()
}
```

Üstteki engine bu interface dışında hiçbir şey bilmez. Her kurum için FIX 4.2/4.4, REST veya WebSocket tabanlı bir implementasyon yazılır; idempotency ve cancel/replace semantiği adapter'da normalize edilir. Mock adapter geliştirme ve UI testleri için anında ACK + simüle fill üretir.

10. Trader UX: workspace ve DOM

Problemin yarısı backend değil, trader UX. Ana ekran: sol watchlist + market depth, ortada chart (entry/TP/SL çizgileri sürüklenabilir) ve blotter (orders / positions / trades), sağda order ticket (Limit · Market · Stop · Advanced) ve advanced panel (TP, SL, trailing, OCO, scale-out toggle'ları).

THYAO 312.50 +1.42%	Position +2,500	P/L +₺18,240
WATCHLIST THYAO GARAN AKBNK ASELS TUPRS	CHART _____ TP _____ ENTRY _____ SL	ORDER ENTRY BUY SELL Qty [2500] Price[..] ☐ Limit ☐ Market ☒ Adv.
MARKET DEPTH 312.60 4,230 312.55 8,120	ORDERS / POSITIONS / TRADES THYAO BUY 2500 FILLED GARAN SELL 1000 WORKING	ADVANCED ☒ TP ☒ SL ☒ Trailing ☒ OCO ☒ Scale Out

DOM / ladder trading zorunlu: fiyat satırına tek tık ile emir, emri mouse ile 312.50 → 312.60 sürükleyince backend cancel/replace yapar. Profesyonel trader için hız buradan gelir.

SELL QTY	PRICE	BUY QTY
12,540	312.80	
8,230	312.75	
18,430	312.70	
	312.60	7,450
	312.55	14,220
	312.50	21,450

11. Teknoloji yığını

Bileşen	Seçim	Gerekçe
Web	Next.js + Tailwind	TradingView Lightweight Charts / custom DOM component; WebSocket ile realtime
Mobil	Expo React Native	Pozisyon/emir takibi, hızlı SL/TP ekleme, bildirim; aynı zod şemaları
API / Engine	NestJS (Fastify)	Event-driven order engine + WS gateway; Next/Expo ile tip paylaşımı. Fon copilot'taki Python gerekçesi burada geçerli değil
Şema	zod (@ems/shared)	Tek kaynak: request validation + frontend tipleri
DB	PostgreSQL + TimescaleDB	order_tree / order_leg / order_event / fill; tick geçmişi
Hot state	Redis	Idempotency key, aktif emir cache, rate limit
Event bus	NATS JetStream	Order event log, replay, fill dağıtımı, tick fan-out
Broker bağlantı	FIX / REST / WS adapter	Kurum spec'ine göre; FIX için QuickFIX tabanlı ayrı gateway servisi seçenek

Ne zaman Go/Rust/Java?

Latency SLA'sı tek haneli milisaniyeye inerse veya FIX session yönetimi ağırlaşırsa yalnızca broker gateway servisi ayrı dilde yazılır; engine ve API Nest'te kalır.

12. Kurulan iskelet — Desktop/bist-ems

pnpm + Turborepo monorepo olarak oluşturuldu:

Yol	İçerik
packages/shared	AdvancedOrderRequest, OrderLeg, OrderStatus, Condition zod şemaları; NATS subject sözleşmesi; FillEvent
packages/broker-adapters	BrokerAdapter interface + MockBrokerAdapter (300 ms sonra tam fill simülasyonu)
apps/api/src/orders	OrderStateMachine (geçiş tablosu), OrderEngineService (root/child aktivasyonu, OCO iptali, kısmi fill'de child qty düşürme), OrdersController
apps/api/src/broker	BrokerRegistry — adapter kaydı ve fill fan-out
apps/api/src/risk	PreTradeRiskService — max qty; bakiye, fat-finger, günlük zarar limiti TODO
apps/api/src/market-data	MarketDataService — tick cache, trigger engine iskeleti
infra/docker-compose.yml	TimescaleDB, Redis, NATS JetStream
apps/web, apps/mobile	README + create komutları (network gerektirdiği için lokalde çalıştırılacak)
docs/ARCHITECTURE.md	Katmanlar, sentetik emir modeli, sıradaki adımlar

Engine şimdilik in-memory; README.md 'de mock broker ile test edilebilecek bracket emir JSON'u var. Not: sandbox'tan git init tamamlanamadı, bozuk .git dizini _to_delete/ altına taşındı — o klasörü silip lokalde

git init yapmak yeterli.

13. Riskler ve uyum

- **Emir iletim aracılığı:** Ürün, kurumun kendi OMS'i üzerinden ve kurumun müşterisi adına emir ilettiği için lisans kurumun lisansdır; ancak pre-trade risk kontrolleri, emir kayıtları ve audit log'un SPK/BİAŞ gözetim gereklilikleriyle uyumlu olması şarttır. Gözetimden kaçırma amaçlı hiçbir özellik kapsam dışıdır.
- **Idempotency ve çift emir:** Adapter ACK gelmeden retry yapılmaz; her leg tek `emsOrderId` ile gönderilir; kurum tarafında client order id eşlemesi zorunlu.
- **Kısmi fill semantiği:** Child qty'ler parent'ın gerçekleşen miktarına göre dinamik; stop qty her fill'de yeniden hesaplanır.
- **Feed bağımlılığı:** Trigger engine tick kalitesine bağlı; feed kesintisinde STOP leg'lerin davranışı (bekle / kurum stop'una fallback) ürün kararı olarak tanımlanmalı.
- **Bağlantı kopması:** Sunucu tarafı state sayesinde tarayıcı kapanması zinciri etkilemez; NATS replay ile process restart'ta state yeniden kurulur.

14. Yol haritası

#	Adım	Çıktı
1	Persist	Prisma şeması: order_tree, order_leg, order_event, fill; restart sonrası state recovery
2	Event bus + WS	NATS publish/consume; WebSocket gateway ile order state, fill, tick push
3	Trigger engine	STOP / TRAILING / CONDITIONAL değerlendirme; market-data entegrasyonu
4	İlk gerçek adapter	Müşterinin kurumu — API/FIX spec'i alınacak; sandbox ortamı
5	Web workspace	DOM/ladder, order ticket, zincir emir builder, blotter, chart overlay
6	Algo slicer	TWAP / VWAP / iceberg; POV için hacim feed'i
7	Pre-trade risk tam set	Bakiye, pozisyon limiti, fat-finger, günlük zarar, sembol yasağı; kurum bazlı config
8	Mobil	Expo: pozisyon/emir takibi, hızlı SL/TP, push bildirim
9	Multi-broker	İkinci kurum adapter'ı ile white-label kanıtı

Önerilen ilk teslimat: müşteri şikâyeti UI'dan başladığı için DOM/ladder + zincir emir builder ekranı, mock broker üzerinde çalışır demo olarak.